```
/**
 * pages/scheduled-reports.tsx
 *
 * Changes from original:
 *
 * [Bug fix]  Delete button now wrapped in AlertDialog — fires immediately
 *            in the original, causing accidental deletes.
 *
 * [Bug fix]  toggleMutation now has onError handler with toast + refetch.
 *            Previously a failed toggle would snap the Switch back silently.
 *
 * [Bug fix]  runNowMutation now polls the schedule's lastRunAt to detect
 *            completion and shows a follow-up toast. Previously the user got
 *            "Your report is being generated" and then nothing.
 *
 * [Rec #8]   Preview button opens a modal showing what the report looks like
 *            before any scheduled send — prevents first-send surprises.
 *
 * [UX fix]   Button row gets flex-wrap so it doesn't overflow on narrow cards.
 */

import { useState, useRef } from 'react';
import { useQuery, useMutation } from '@tanstack/react-query';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { queryClient, apiRequest } from '@/lib/queryClient';
import {
  Card, CardContent, CardHeader, CardTitle, CardDescription,
} from '@/components/ui/card';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Badge } from '@/components/ui/badge';
import { Switch } from '@/components/ui/switch';
import {
  Select, SelectContent, SelectItem, SelectTrigger, SelectValue,
} from '@/components/ui/select';
import {
  Dialog, DialogContent, DialogHeader, DialogTitle,
  DialogTrigger, DialogFooter, DialogDescription,
} from '@/components/ui/dialog';
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription,
```

```
  AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
  AlertDialogTrigger,
} from '@/components/ui/alert-dialog';
import {
  Form, FormField, FormItem, FormLabel, FormControl, FormMessage,
} from '@/components/ui/form';
import { ScrollArea } from '@/components/ui/scroll-area';
import { Separator } from '@/components/ui/separator';
import { useToast } from '@/hooks/use-toast';
import {
  Plus, Calendar, Mail, FileText, Clock,
  Play, Trash2, Eye, Loader2, CheckCircle2,
} from 'lucide-react';
import type { ReportSchedule } from '@shared/schema/scheduled-reports';

// ─── Constants ──────────────────────────────────────────────────────────────

const REPORT_TYPES = [
  { id: 'fleet_health',     name: 'Fleet Health Summary' },
  { id: 'maintenance_due',  name: 'Maintenance Due Report' },
  { id: 'inventory_status', name: 'Inventory Status Report' },
  { id: 'crew_compliance',  name: 'Crew Compliance Report' },
  { id: 'cost_summary',     name: 'Cost Summary Report' },
] as const;

const FREQUENCIES = [
  { id: 'daily',   name: 'Daily',   cron: '0 8 * * *' },
  { id: 'weekly',  name: 'Weekly',  cron: '0 8 * * 1' },
  { id: 'monthly', name: 'Monthly', cron: '0 8 1 * *' },
] as const;

const FORMATS = [
  { id: 'pdf',  name: 'PDF'  },
  { id: 'csv',  name: 'CSV'  },
  { id: 'json', name: 'JSON' },
] as const;

// ─── Schema ─────────────────────────────────────────────────────────────────

const createScheduleFormSchema = z.object({
  name:        z.string().min(1, 'Name is required').max(100),
  reportType:  z.enum(['fleet_health', 'maintenance_due', 'inventory_status', 'cr
  frequency:   z.enum(['daily', 'weekly', 'monthly']),
  format:      z.enum(['pdf', 'csv', 'json']),
  recipients:  z.string().min(1, 'At least one recipient is required'),
  enabled:     z.boolean(),
});
```

```
type CreateScheduleForm = z.infer<typeof createScheduleFormSchema>;

// ─── Report Preview Modal ──────────────────────────────────────────────
//
// Recommendation #8 — Preview before first send.
// Triggers a one-off generation and shows the result in a scrollable modal.

interface PreviewModalProps {
  schedule: ReportSchedule;
  getReportTypeName: (id: string) => string;
}

function PreviewModal({ schedule, getReportTypeName }: PreviewModalProps) {
  const { toast } = useToast();
  const [previewData, setPreviewData] = useState<{
    content: string;
    generatedAt: string;
  } | null>(null);
  const [isGenerating, setIsGenerating] = useState(false);
  const [isOpen, setIsOpen] = useState(false);

  const handleOpen = async () => {
    setIsOpen(true);
    if (previewData) return; // Already generated this session
    setIsGenerating(true);
    try {
      const res = await apiRequest('POST', `/api/scheduled-reports/schedules/${sc
      setPreviewData(res);
    } catch {
      toast({
        title: 'Preview failed',
        description: 'Could not generate a preview. Check report configuration.',
        variant: 'destructive',
      });
      setIsOpen(false);
    } finally {
      setIsGenerating(false);
    }
  };

  return (
    <Dialog open={isOpen} onOpenChange={setIsOpen}>
      <DialogTrigger asChild>
        <Button
          size="sm"
          variant="outline"
```

```
          onClick={handleOpen}
          data-testid={`button-preview-${schedule.id}`}
        >
          <Eye className="w-3 h-3 mr-1" />
          Preview
        </Button>
      </DialogTrigger>

      <DialogContent className="sm:max-w-2xl max-h-[80vh] flex flex-col">
        <DialogHeader>
          <DialogTitle>Report Preview — {schedule.name}</DialogTitle>
          <DialogDescription>
            {getReportTypeName(schedule.reportType)} · {schedule.format.toUpperCa
            {(schedule.recipients as string[])?.length ?? 0} recipients
          </DialogDescription>
        </DialogHeader>

        <div className="flex-1 min-h-0">
          {isGenerating ? (
            <div className="flex flex-col items-center justify-center py-12 gap-3
              <Loader2 className="h-6 w-6 animate-spin text-muted-foreground" />
              <p className="text-sm text-muted-foreground">Generating preview…</p
            </div>
          ) : previewData ? (
            <ScrollArea className="h-[50vh] rounded-md border p-4">
              <pre className="text-sm font-mono whitespace-pre-wrap text-foregrou
                {previewData.content}
              </pre>
            </ScrollArea>
          ) : null}
        </div>

        {previewData && (
          <div className="flex items-center gap-2 pt-2 border-t text-xs text-mute
            <CheckCircle2 className="h-3.5 w-3.5 text-emerald-500" />
            Preview generated {new Date(previewData.generatedAt).toLocaleTimeStri
          </div>
        )}
      </DialogContent>
    </Dialog>
  );
}

// ─── Main Page ──────────────────────────────────────────────────────────────

export default function ScheduledReports() {
  const { toast } = useToast();
```

```
const [isCreateOpen, setIsCreateOpen] = useState(false);
// Track which schedule IDs are currently polling for run completion
const runPollingRefs = useRef<Map<string, ReturnType<typeof setInterval>>>(new

const form = useForm<CreateScheduleForm>({
  resolver: zodResolver(createScheduleFormSchema),
  defaultValues: {
    name:       '',
    reportType: 'fleet_health',
    frequency:  'weekly',
    format:     'pdf',
    recipients: '',
    enabled:    true,
  },
});

const { data: schedulesResponse, isLoading } = useQuery<{ data: ReportSchedule[
  queryKey: ['/api/scheduled-reports/schedules'],
});

const schedules = schedulesResponse?.data ?? [];

// ── Create ──────────────────────────────────────────────────────────────────

const createMutation = useMutation({
  mutationFn: async (data: CreateScheduleForm) => {
    const cronExpr = FREQUENCIES.find(f => f.id === data.frequency)?.cron ?? '0
    const recipientList = data.recipients.split(',').map(r => r.trim()).filter(
    return apiRequest('POST', '/api/scheduled-reports/schedules', {
      name:           data.name,
      reportType:     data.reportType,
      frequency:      data.frequency,
      cronExpression: cronExpr,
      timezone:       Intl.DateTimeFormat().resolvedOptions().timeZone ?? 'UTC'
      format:         data.format,
      recipients:     recipientList,
      vesselIds:      null,
      enabled:        data.enabled,
    });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/scheduled-reports/schedule
    setIsCreateOpen(false);
    form.reset();
    toast({ title: 'Schedule created', description: 'Your report schedule has b
  },
  onError: () => {
```

```
    toast({ title: 'Error', description: 'Failed to create schedule.', variant:
  },
});

// ── Toggle enabled ──────────────────────────────────────────────────────
// Bug fix: added onError handler

const toggleMutation = useMutation({
  mutationFn: async ({ id, enabled }: { id: string; enabled: boolean }) => {
    return apiRequest('PATCH', `/api/scheduled-reports/schedules/${id}`, { enab
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/scheduled-reports/schedule
  },
  onError: (_, { id }) => {
    // Refetch so the Switch snaps back to the server value visually
    queryClient.invalidateQueries({ queryKey: ['/api/scheduled-reports/schedule
    toast({
      title: 'Failed to update schedule',
      description: 'Could not toggle the schedule. Please try again.',
      variant: 'destructive',
    });
  },
});

// ── Run Now ─────────────────────────────────────────────────────────────
// Bug fix: polls lastRunAt to confirm completion and shows a follow-up toast.

const runNowMutation = useMutation({
  mutationFn: async (id: string) => {
    return apiRequest('POST', `/api/scheduled-reports/schedules/${id}/run`);
  },
  onSuccess: (_, id) => {
    toast({ title: 'Report generating…', description: 'We\'ll notify you when i

    // Poll every 5s for up to 2 minutes for the lastRunAt to update
    const start = Date.now();
    const poll = setInterval(async () => {
      if (Date.now() - start > 120_000) {
        clearInterval(poll);
        runPollingRefs.current.delete(id);
        return;
      }
      await queryClient.invalidateQueries({ queryKey: ['/api/scheduled-reports/
      const fresh = queryClient.getQueryData<{ data: ReportSchedule[] }>(['/api
      const updated = fresh?.data.find(s => s.id === id);
      if (updated?.lastRunAt && new Date(updated.lastRunAt) > new Date(start))
```

```
        clearInterval(poll);
        runPollingRefs.current.delete(id);
        toast({
          title: 'Report ready',
          description: `"${updated.name}" has been generated and sent.`,
        });
      }
    }, 5000);

    runPollingRefs.current.set(id, poll);
  },
  onError: () => {
    toast({ title: 'Error', description: 'Failed to run report.', variant: 'des
  },
});

// ── Delete ──────────────────────────────────────────────────────────────
// Bug fix: wrapped in AlertDialog (previously fired immediately)

const deleteMutation = useMutation({
  mutationFn: async (id: string) => {
    return apiRequest('DELETE', `/api/scheduled-reports/schedules/${id}`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/scheduled-reports/schedule
    toast({ title: 'Schedule deleted', description: 'Your report schedule has b
  },
  onError: () => {
    toast({ title: 'Error', description: 'Failed to delete schedule.', variant:
  },
});

// ── Helpers ─────────────────────────────────────────────────────────────

const handleSubmit = (data: CreateScheduleForm) => createMutation.mutate(data);

const getReportTypeName = (id: string) =>
  REPORT_TYPES.find(t => t.id === id)?.name ?? id;

const formatDate = (date: string | Date | null) => {
  if (!date) return 'Never';
  return new Date(date).toLocaleString();
};

// ────────────────────────────────────────────────────────────────────────

return (
```

```
<div className="container mx-auto p-6 space-y-6" data-testid="page-scheduled-
  <div className="flex items-center justify-between flex-wrap gap-4">
    <div>
      <h1 className="text-2xl font-bold" data-testid="text-page-title">Schedu
      <p className="text-muted-foreground">Automate report generation and del
    </div>

    {/* ── Create schedule dialog ──────────────────────────────────────── */}
    <Dialog open={isCreateOpen} onOpenChange={setIsCreateOpen}>
      <DialogTrigger asChild>
        <Button data-testid="button-create-schedule">
          <Plus className="w-4 h-4 mr-2" />
          New Schedule
        </Button>
      </DialogTrigger>
      <DialogContent className="sm:max-w-md">
        <DialogHeader>
          <DialogTitle>Create Report Schedule</DialogTitle>
          <DialogDescription>Set up automated report generation and delivery.
        </DialogHeader>
        <Form {...form}>
          <form onSubmit={form.handleSubmit(handleSubmit)} className="space-y
            <FormField
              control={form.control}
              name="name"
              render={({ field }) => (
                <FormItem>
                  <FormLabel>Schedule Name</FormLabel>
                  <FormControl>
                    <Input {...field} data-testid="input-schedule-name" place
                  </FormControl>
                  <FormMessage />
                </FormItem>
              )}
            />
            <FormField
              control={form.control}
              name="reportType"
              render={({ field }) => (
                <FormItem>
                  <FormLabel>Report Type</FormLabel>
                  <Select onValueChange={field.onChange} value={field.value}>
                    <FormControl>
                      <SelectTrigger data-testid="select-report-type">
                        <SelectValue />
                      </SelectTrigger>
                    </FormControl>
```

```
            <SelectContent>
              {REPORT_TYPES.map(t => <SelectItem key={t.id} value={t.
            </SelectContent>
          </Select>
          <FormMessage />
        </FormItem>
      )}
    />
    <div className="grid grid-cols-2 gap-4">
      <FormField
        control={form.control}
        name="frequency"
        render={({ field }) => (
          <FormItem>
            <FormLabel>Frequency</FormLabel>
            <Select onValueChange={field.onChange} value={field.value
              <FormControl>
                <SelectTrigger data-testid="select-frequency">
                  <SelectValue />
                </SelectTrigger>
              </FormControl>
              <SelectContent>
                {FREQUENCIES.map(f => <SelectItem key={f.id} value={f
              </SelectContent>
            </Select>
            <FormMessage />
          </FormItem>
        )}
      />
      <FormField
        control={form.control}
        name="format"
        render={({ field }) => (
          <FormItem>
            <FormLabel>Format</FormLabel>
            <Select onValueChange={field.onChange} value={field.value
              <FormControl>
                <SelectTrigger data-testid="select-format">
                  <SelectValue />
                </SelectTrigger>
              </FormControl>
              <SelectContent>
                {FORMATS.map(f => <SelectItem key={f.id} value={f.id}
              </SelectContent>
            </Select>
            <FormMessage />
          </FormItem>
```

```
              )}
            />
          </div>
          <FormField
            control={form.control}
            name="recipients"
            render={({ field }) => (
              <FormItem>
                <FormLabel>Recipients (comma-separated emails)</FormLabel>
                <FormControl>
                  <Input {...field} data-testid="input-recipients" placehol
                </FormControl>
                <FormMessage />
              </FormItem>
            )}
          />
          <DialogFooter>
            <Button type="button" variant="outline" onClick={() => setIsCre
              Cancel
            </Button>
            <Button type="submit" disabled={createMutation.isPending} data-
              {createMutation.isPending ? 'Creating…' : 'Create Schedule'}
            </Button>
          </DialogFooter>
        </form>
      </Form>
    </DialogContent>
  </Dialog>
</div>

{/* ── Loading skeletons ──────────────────────────────────────────── *
{isLoading ? (
  <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
    {[1, 2, 3].map(i => (
      <Card key={i} className="animate-pulse">
        <CardHeader className="space-y-2">
          <div className="h-5 bg-muted rounded w-3/4" />
          <div className="h-4 bg-muted rounded w-1/2" />
        </CardHeader>
        <CardContent>
          <div className="h-4 bg-muted rounded w-full mb-2" />
          <div className="h-4 bg-muted rounded w-2/3" />
        </CardContent>
      </Card>
    ))}
  </div>
) : schedules.length === 0 ? (
```

```
  /* ── Empty state ──────────────────────────────────────────────────── */
  <Card className="text-center py-12">
    <CardContent>
      <Calendar className="w-12 h-12 mx-auto mb-4 text-muted-foreground" />
      <h3 className="text-lg font-medium mb-2">No scheduled reports</h3>
      <p className="text-muted-foreground mb-4">Create your first automated
      <Button onClick={() => setIsCreateOpen(true)} data-testid="button-cre
        <Plus className="w-4 h-4 mr-2" />
        Create Schedule
      </Button>
    </CardContent>
  </Card>
) : (

  /* ── Schedule cards ──────────────────────────────────────────────── *
  <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
    {schedules.map(schedule => (
      <Card key={schedule.id} data-testid={`card-schedule-${schedule.id}`}>
        <CardHeader>
          <div className="flex items-start justify-between gap-2">
            <div className="flex-1 min-w-0">
              <CardTitle className="text-base truncate" data-testid={`text-
                {schedule.name}
              </CardTitle>
              <CardDescription className="truncate">
                {getReportTypeName(schedule.reportType)}
              </CardDescription>
            </div>
            <Switch
              checked={schedule.enabled}
              onCheckedChange={checked =>
                toggleMutation.mutate({ id: schedule.id, enabled: checked }
              }
              data-testid={`switch-enabled-${schedule.id}`}
            />
          </div>
        </CardHeader>

        <CardContent className="space-y-4">
          <div className="flex flex-wrap gap-2">
            <Badge variant="secondary">
              <Clock className="w-3 h-3 mr-1" />
              {schedule.frequency}
            </Badge>
            <Badge variant="outline">
              <FileText className="w-3 h-3 mr-1" />
```

```
    {schedule.format.toUpperCase()}
  </Badge>
  <Badge variant="outline">
    <Mail className="w-3 h-3 mr-1" />
    {(schedule.recipients as string[])?.length ?? 0}
  </Badge>
</div>

<div className="text-sm text-muted-foreground space-y-1">
  <p>Last run: {formatDate(schedule.lastRunAt)}</p>
  <p>Next run: {formatDate(schedule.nextRunAt)}</p>
</div>

{/* Action row — flex-wrap so it doesn't overflow narrow cards */
<div className="flex gap-2 flex-wrap">
  {/* Run Now */}
  <Button
    size="sm"
    variant="outline"
    onClick={() => runNowMutation.mutate(schedule.id)}
    disabled={runNowMutation.isPending}
    data-testid={`button-run-${schedule.id}`}
  >
    {runNowMutation.isPending ? (
      <Loader2 className="w-3 h-3 mr-1 animate-spin" />
    ) : (
      <Play className="w-3 h-3 mr-1" />
    )}
    Run Now
  </Button>

  {/* Preview — Rec #8 */}
  <PreviewModal schedule={schedule} getReportTypeName={getReportT

  {/* Delete — wrapped in AlertDialog (bug fix) */}
  <AlertDialog>
    <AlertDialogTrigger asChild>
      <Button
        size="sm"
        variant="ghost"
        disabled={deleteMutation.isPending}
        data-testid={`button-delete-${schedule.id}`}
      >
        <Trash2 className="w-3 h-3" />
      </Button>
    </AlertDialogTrigger>
    <AlertDialogContent>
```

```
                    <AlertDialogHeader>
                      <AlertDialogTitle>Delete schedule?</AlertDialogTitle>
                      <AlertDialogDescription>
                        <strong>&ldquo;{schedule.name}&rdquo;</strong> will be
                        will no longer run. This cannot be undone.
                      </AlertDialogDescription>
                    </AlertDialogHeader>
                    <AlertDialogFooter>
                      <AlertDialogCancel>Cancel</AlertDialogCancel>
                      <AlertDialogAction
                        onClick={() => deleteMutation.mutate(schedule.id)}
                        className="bg-destructive text-destructive-foreground h
                      >
                        Delete
                      </AlertDialogAction>
                    </AlertDialogFooter>
                  </AlertDialogContent>
                </AlertDialog>
              </div>
            </CardContent>
          </Card>
        ))}
      </div>
    )}
  </div>
 );
}
```